

4-3-6 Options Pattern 選項模式

前面應用程式直接存取組態值會造成緊密耦合的問題，為避免這問題，ASP.NET Core 使用 Options Pattern 選項模式抽象化背後的組態系統，將組態資料先繫結到「類別」，應用程式再存取該類別，使得應用程式不直接相依組態系統。日後組態資料結構有任何的調整，應用程式也不必連動做修改，唯一需要調整的是 Options 類別的設定。

Options 類別有兩個要求：

1. 必須為非抽象類別
2. 建構函式必須為 public 且無參數

選項模式建立及使用步驟有四：

1. 建立組態資料，並載入組態系統中
2. 建立 Options 類別
3. 在 Program.cs 中以 builder.**Services.Configure**<T> 方法註冊 Options 類別
4. 在 Controller / View / Services 中存取 Options 類別

範例 4-1 透過 Options 選項模式讀取組態設定

以前面 appsettings.json 組態的 Developer 區段設定值為例，不直接存取組態，而是透過 Options 選項模式讀取組態設定。



圖 4-9 透過 Options 選項類別讀取組態設定

step01 於 appsettings.json 組態中建立 Developer 開發者資料

📄 appsettings.json

```
{
  ...
  "Developer": {
    "Name": "聖殿祭司",
    "Email": "dotnetcool@gmail.com",
    "Website": "https://www.codemagic.com.tw"
  }
}
```

step02 建立 DeveloperOptions 類別，三個屬性名稱須與組態 Key 名稱對映

📄 Options/DeveloperOptions.cs

```
public class DeveloperOptions
{
    public string Name { get; set; }
    public string Email { get; set; }
    public string Website { get; set; }
}
```

step 03 在 Program.cs 的 DI Container 註冊 DeveloperOptions 類別，繫結其對應的組態設定

 program.cs

```
//在 DI Container 中註冊 DeveloperOptions 類別
builder.Services.Configure<DeveloperOptions>(options =>
    builder.Configuration.GetSection("Developer").Bind(options));
```

DeveloperOptions 類別

讀取組態區段

將組態繫結至 DeveloperOptions 類別

step 04 在 View 中注入 DeveloperOptions 選項類別實例

 Views/Fundamentals/AccessDeveloperOptions.cshtml

```
@using Microsoft.Extensions.Options
@Inject IOptionsMonitor<DeveloperOptions> developerOptions

@{
    var devOptions = developerOptions.CurrentValue;
}

<ul>
    <li>Name : @devOptions.Name</li>
    <li>Email : @devOptions.Email</li>
    <li>Website : @devOptions.Website</li>
</ul>
```

IOptionsMonitor<TOptions>

須呼叫 CurrentValue 屬性

讀取 Options 類別屬性

4-3-7 Environment 環境

ASP.NET Core 執行時會讀取 ASPNETCORE_ENVIRONMENT 環境變數，判斷它是 Development、Staging 或 Production 環境，依序代表開發、預備和生產環境，並依環境的不同而載入對應程式或設定。